



Winning Space Race with Data Science

Asma Boufaden

23/01/2026



Outline

- Executive Summary
- Introduction
- Methodology
- Results
- Conclusion
- Appendix

Executive Summary

Objective: Predict Falcon 9 first-stage landing success to assess cost savings and competitiveness.

Methodology

- Data collected via SpaceX REST API + Wikipedia scraping.
- Data wrangling: handled missing values, created target *Class* (success/failure).
- Feature engineering + standardization.
- EDA with SQL & visualizations (Matplotlib, Seaborn).
- Interactive tools: Folium maps, Plotly Dash dashboard.
- Predictive models: Logistic Regression, SVM, Decision Tree, KNN - (GridSearchCV).

Results

- Validation accuracy: Decision Tree best (0.875).
- Test accuracy: all models ~0.833, same confusion matrix.
- Models perfect at predicting success, weaker at predicting failure.

Key insights

- Best site: **KSC LC-39A**.
- Reliable payloads: < 6000 kg
- Orbit effect: **LEO > GTO**.
- Booster effect: **FT, B4, B5 outperform earlier versions**.

Introduction

- **Project background and context**

SpaceX has gained worldwide attention for a series of historic milestones. It is the only private company ever to return a spacecraft from low-earth orbit, which it first accomplished in December 2010. SpaceX advertises Falcon 9 rocket launches on its website with a cost of 62 million dollars whereas other providers cost upward of 165 million dollars each, much of the savings is because Space X can reuse the first stage.

Therefore, if we can determine if the first stage will land, we can determine the cost of a launch.

- **Problems you want to find answers**

We will predict if the Falcon 9 first stage will land successfully. This information can be used if an alternate company wants to bid against SpaceX for a rocket launch and hence reducing its launch costs.

Section 1

Methodology

Methodology

The project analyzes **SpaceX launch data** to understand factors influencing mission success and predict if the Falcon 9 first stage will land successfully.

- **Data collection methodology:** Data sets collected using the **SPACE X REST API** and performing web scraping from a Wikipedia page titled "[List of Falcon 9 and Falcon Heavy launches](#)".
- **Perform data wrangling:** Handled missing values, Remove Noise Types, Exploratory Counts, Convert Outcomes to Labels and creating our target column "Class": 1 for success landing or 0 for failure.
- **Perform Data Processing:** Feature Engineering (dummy variables, OneHotEncoder).
- **Perform exploratory data analysis (EDA) using visualization and SQL:** SQL queries used for filtering, extracting unique launch sites, payloads statistics, identifying outcomes, and ranking results. Visualization tools: Matplotlib, Seaborn.
- **Perform interactive visual analytics using Folium and Plotly Dash:** Folium for Mapped launch sites geographically. Plotly Dash to build interactive dashboards with dropdowns, sliders, pie charts, scatter plots.
- **Perform predictive analysis using classification models:** Logistic Regression, SVM, Decision Tree, KNN models used to predict landing success based on payload mass, orbit, site, etc. Dataset splitted into training and test sets and standardization of data. Applied hyperparameter tuning (GridSearchCV, cv=10). Models evaluated using accuracy and confusion matrix.

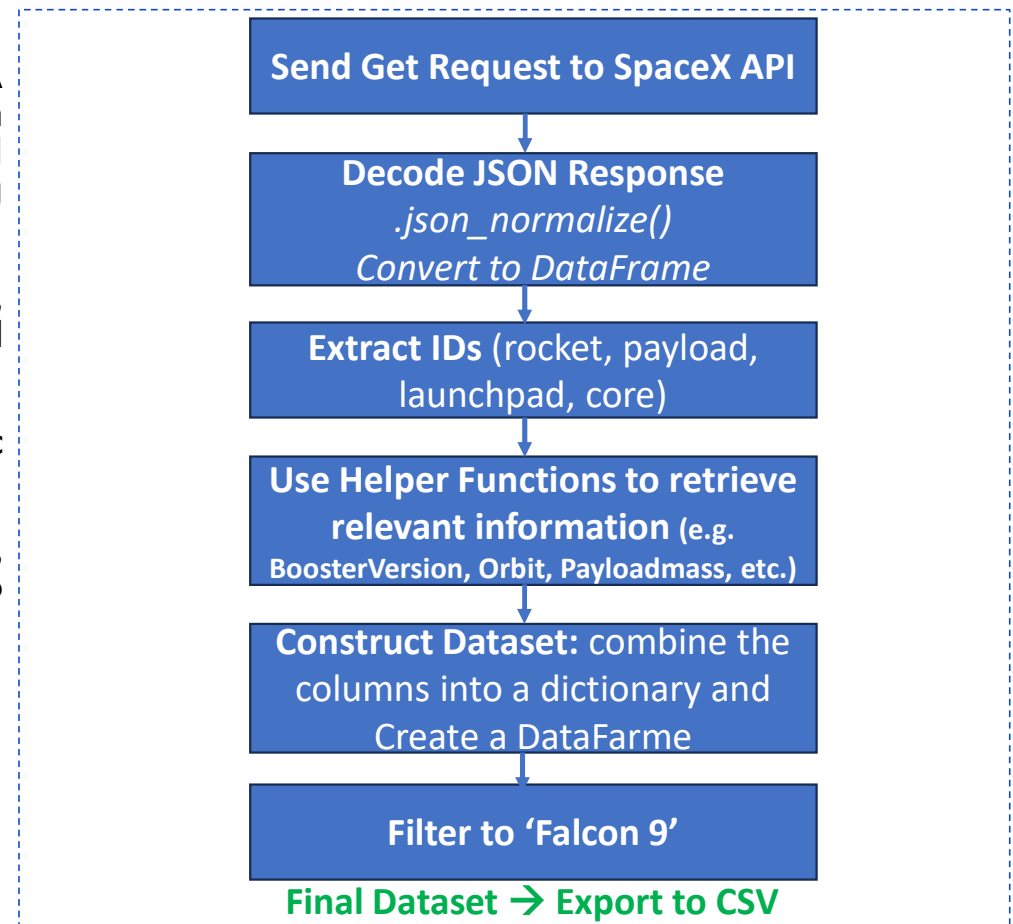
Data Collection

- Data sets collected using the **SPACE X REST API**.
- Data collected performing web scraping to collect Falcon 9 historical launch records from a Wikipedia page titled "[List of Falcon 9 and Falcon Heavy launches](#)".

These two methods allow us to build a rich and structured dataset to predict Falcon 9 first stage landings.

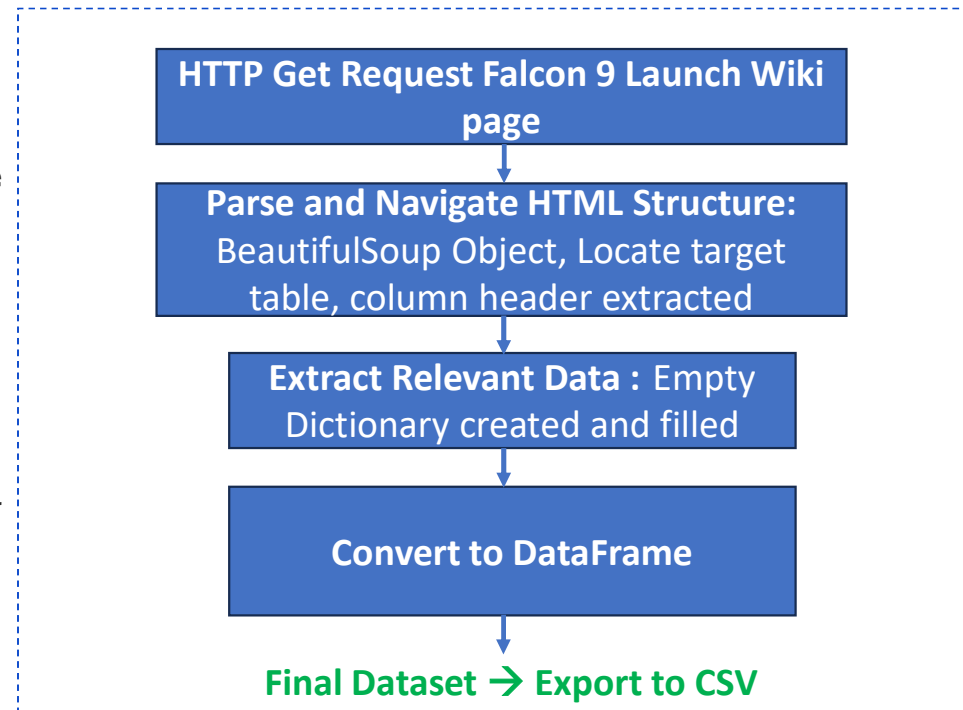
Data Collection – SpaceX API

- Dataset collected using the **SpaceX REST API**. A **GET request** was made to retrieve launch data in JSON format and the JSON response was **decoded** and converted into a **Pandas DataFrame** using `.json_normalize()`.
- Since many columns contained only **IDs** (e.g., rocket, payload, launchpad, cores), additional API calls were made to retrieve detailed information.
- Helper functions were defined to extract specific attributes from these IDs.
- Finally, the data was **combined** into a dictionary, converted into a DataFrame, and **filtered** to include only **Falcon 9** launches.
- **GitHub URL:** [1.Capstone_jupyter-labs-spacex-data-collection-api](https://github.com/1.Capstone/jupyter-labs-spacex-data-collection-api)



Data Collection - Scraping

- Data collected performing **web scraping** - HTTP GET method to request the Falcon9 Launch HTML page.
- **BeautifulSoup Object** created to parse and navigate HTML structure.
- Target HTML table Falcon 9 launch records located, and column headers extracted from <th> elements.
- Empty Dictionary created with column headers as keys.
- Parse table rows and fill Dictionary with launch data using helper functions.
- Convert to Pandas DataFrame and Export to CSV.
- **GitHub URL:** [Capstone_jupyter-labs-webscraping.ipynb](#)



Data Wrangling

1. Handling Missing Value & Noise Types

- Calculated the mean of Payload mass and replace np.nan values with the mean.
- Retained None values in LandingPad to indicate cases where no landing pad was used.
- Do not retain unexpected annotations and other types of noises (reference links, etc.) during parsing process.

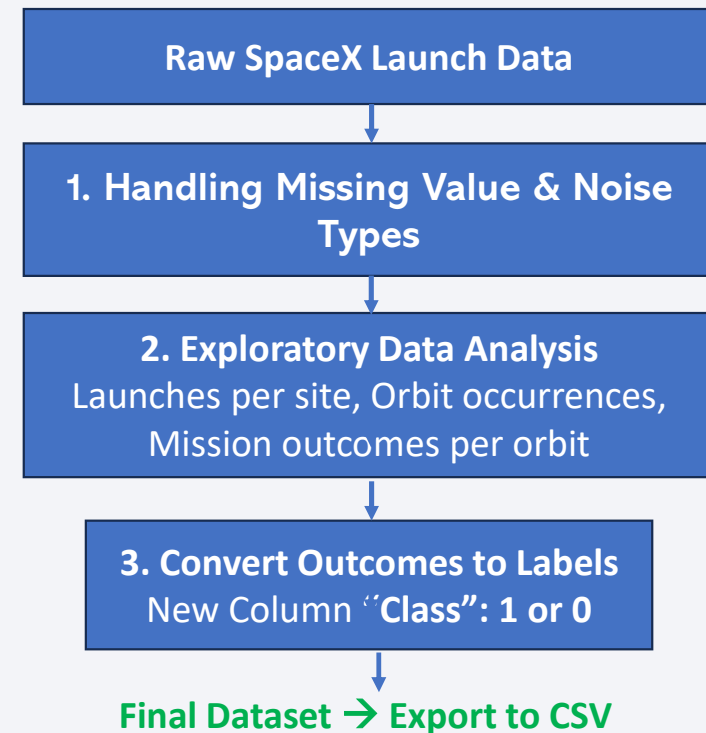
2. Identify Data Types & Exploratory Counts

- Identify Numerical and Categorical columns.
- Count number of launches per site, orbit (excluding GTO), orbit mission outcomes.

3. Convert Outcomes into Training Labels

- Defined successful (e.g. True Ocean, True RTLS, True ASDS) and unsuccessful outcomes (e.g. False Ocean, False RTLS, etc.).
- Generated a new column 'Class' → 1 if successful, 0 if unsuccessful.

- **GitHub URL:** [Capstone labs-jupyter-spacex-Data wrangling.ipynb](https://github.com/AsmaBoufaden/Capstone_labs-jupyter-spacex-Data-wrangling.ipynb)



EDA with SQL

This summary shows Exploratory Data Analysis with SQL querying steps performed: extracting unique values, filtering records, payloads statistics, identifying outcomes, and ranking results:

- **Task 1: Display the names of the unique launch sites in the space mission** using `SELECT DISTINCT "Launch_Site"`
- **Task 2: Display 5 records where launch sites begin with the string 'CCA'** using `LIKE 'CCA%'` and `LIMIT 5`
- **Task 3: Display the total payload mass carried by boosters launched by NASA (CRS)** using `SUM("PAYLOAD_MASS__KG_")`
- **Task 4: Display average payload mass carried by booster version F9 v1.1** using `AVG("PAYLOAD_MASS__KG_")` with condition `Booster_Version LIKE 'F9 v1.1%'`
- **Task 5: List the date when the first successful landing outcome in ground pad was achieved** using `MIN(Date)` with condition `Landing_Outcome = 'Success (ground pad)'`
- **Task 6: List the names of the boosters which have success in drone ship and have payload mass greater than 4000 but less than 6000** using condition `Landing_Outcome = 'Success (drone ship)'` and `4000 < PAYLOAD_MASS__KG_ < 6000`
- **Task 7: List the total number of successful and failure mission outcomes** using `COUNT` and `GROUP BY Mission_Outcome`
- **Task 8: List all the booster_versions that have carried the maximum payload mass, using a subquery with a suitable aggregate function** using `MAX("PAYLOAD_MASS__KG_")`
- **Task 9: List the records which will display the month names, failure landing_outcomes in drone ship ,booster versions, launch_site for the months in year 2015** using `substr(Date, 6, 2)` for month and condition `Landing_Outcome ='Failure (drone ship)'` AND `substr(Date, 0, 5) ='2015'`
- **Task 10: Rank the count of landing outcomes (such as Failure (drone ship) or Success (ground pad)) between the date 2010-06-04 and 2017-03-20, in descending order** using `COUNT`, condition, `GROUP BY` and `ORDER BY`
- **GitHub URL:** [Capstone_jupyter-labs-eda-sql-coursera_sqlite.ipynb](https://github.com/ibm-space-x-asma-boufaden/capstone_jupyter-labs-eda-sql-coursera_sqlite.ipynb)

EDA with Data Visualization

- **Task 1: Flight Number vs Launch Site (catplot)**
 - *Why:* To show how success rates evolved over time at different launch sites.
 - *Insight:* Success increased with higher flight numbers, and sites had different success histories.
- **Task 2: Payload Mass vs Launch Site (catplot)**
 - *Why:* To compare payload ranges handled by each launch site and see if payload mass influenced success.
 - *Insight:* VAFB handled only lighter payloads (<10000 kg), while CCAFS and KSC managed heavier payloads with successful landings.
- **Task 3: Success Rate by Orbit Type (bar chart)**
 - *Why:* To compare reliability of missions across different orbit types.
 - *Insight:* LEO, PO, ISS, MEO, VLEO had moderate to high success; GTO showed the lowest success rate, highlighting its complexity and ES_L1, GEO, HEO, SSO shower 100% success rate but these missions are fewer in number.
- **Task 4: Flight Number vs Orbit Type (catplot)**
 - *Why:* To analyze how experience (flight number) influenced success for different orbits.
 - *Insight:* Success improved with flight number in LEO and ISS, but GTO remained inconsistent regardless of experience.
- **Task 5: Payload Mass vs Orbit Type (catplot)**
 - *Why:* To examine how payload mass interacts with orbit type and landing success.
 - *Insight:* Heavy payloads were successfully landed in LEO, PO, ISS; GTO missions showed mixed outcomes across payload ranges.
- **Task 6: Success Rate by Year (line chart)**
 - *Why:* To track yearly trends in success rate and visualize SpaceX's progress over time.
 - *Insight:* Success rate rose steadily from 2013, peaked around 2017–2019 (~85–88%), showing technological maturity¹²
- GitHub URL: [Capstone_edadataviz.ipynb](#)

Build an Interactive Map with Folium

- **Map Objects Created**

- **Circle:** Added at each launch site's latitude/longitude coordinates to highlight their location and popup attached to provide site names. Help us to clearly mark and label the geographic positions of launch sites.
 - **Markers:** Placed at each launch site with labels and used to give exact point reference, enhanced with DivIcon for text labels directly on the map.
 - **Outcome Markers (Success/Failure):** Green markers for success (Class=1) and red for failure (Class=0). Added to MarkerCluster to group overlapping markers at same coordinates. Help us to visualize launch success/failure directly on the map to identify which sites had higher/lower success rates.
 - **MousePosition:** To get coordinate for a mouse over a point on the map. Used to capture nearby points of interest (coastlines, highways, railways, cities).
 - **Polylines:** Drawn between a launch site and its closest proximity points (e.g., coastline, city, railway). Visualized distances and spatial relationships between launch sites and nearby infrastructure.
- By combining **circles, markers, clusters, polylines, and interactive tools**, the Folium map provided both **location context** (where sites are) and **performance context** (success vs failure rates), while also enabling **geographical analysis** of proximities that may affect launch outcomes.
- **GitHub URL:** [Capstone lab jupyter launch site location](#)

Build a Dashboard with Plotly Dash

The dashboard combines:

- **Comparative plots (pie chart)** → to compare success rates across all sites or the success/failure ratio for a selected site.
- **Relationship plots (scatter plot)** → to explore how payload mass and booster versions affect success, to answer questions like which payload ranges succeed most or fail most, and which booster version has the highest success rate.
- **Interaction (dropdown + slider)** → to filter and customize the analysis in real time:
 - **Dropdown Menu (Launch Site Selector)**: to filter the dashboard by a specific launch site or view all sites to enable site-level analysis.
 - **Range Slider (Payload Mass Selector)**: to select a payload mass range to filter the scatter plot. Identify whether certain payload ranges correlate with higher or lower success rates.
 - **Callback Functions (Dynamic Updates)**: automatically update the pie chart and scatter plot when dropdown or slider inputs change to ensure real-time interactivity.
- **GitHub URL:** [Capstone_spacex-dash-app.py](#)

Predictive Analysis (Classification)

Model Development Process

1. **Data Preparation:** X from feature engineering (dummy variables, OneHotEncoder), Y = target 'Class' (0/1)

2. **Standardize X:** StandardScaler

3. **Train/Test Split:** Training set (80% of data), Test set (20%, 18 samples).

4. **Logistic Regression:** Tuned hyperparameters with GridSearchCV (C, penalty, solver).

- Validation accuracy: **0.8464**
- Test accuracy: **0.8333**
- Confusion matrix showed 03 false positives as main issue

5. **Support Vector Machine (SVM):** Tuned kernel, C, gamma.

- Validation accuracy: **0.8482**
- Test accuracy: **0.8333**
- Confusion matrix same

6. **Decision Tree:** Tuned criterion, splitter, depth, features, leaf size, split size.

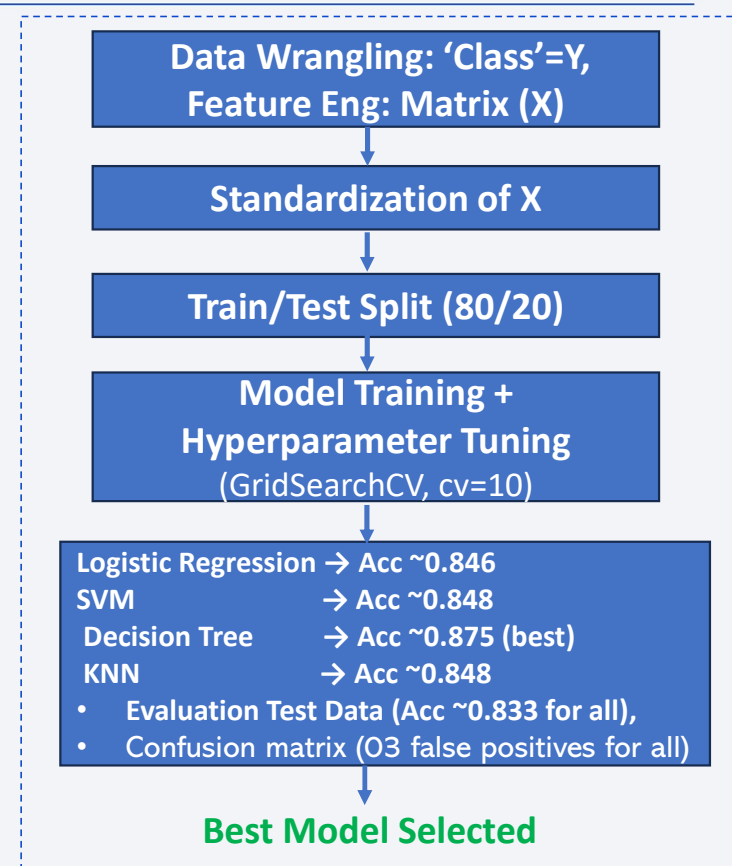
- Validation accuracy: **0.875 (highest)**
- Test accuracy: **0.8333**
- Confusion matrix same

7. **KNN:** Tuned neighbors, algorithm, p

- Validation accuracy: **0.8482**
- Test accuracy: **0.8333**
- Confusion matrix same

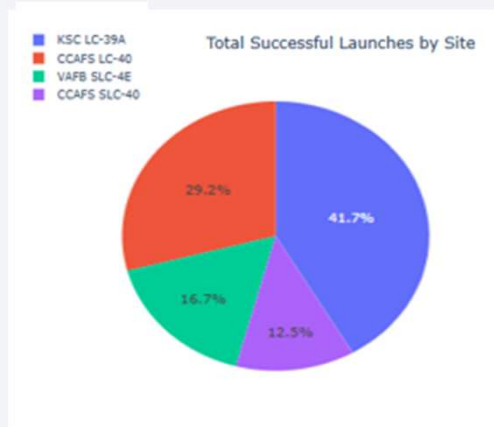
→ **Decision Tree** performed better only in terms of validation accuracy **0.875**. All models are **good at confirming** when the **Falcon 9** first stage will land successfully, but they are **less effective at identifying** when it will fail.

• GitHub URL: [Capstone SpaceX Machine Learning Prediction Part 5.ipynb](#)



Results

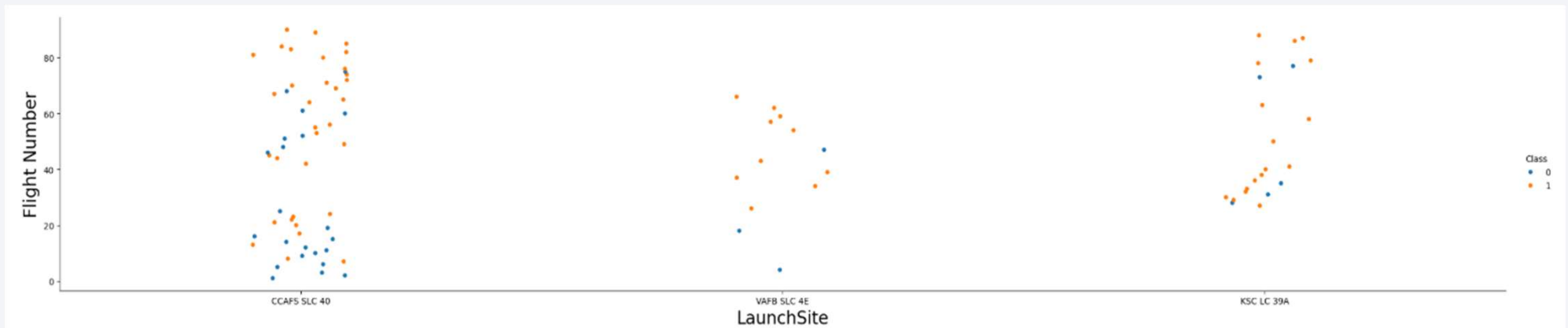
- **Best performing site:** KSC LC-39A (highest success rate).
- **Payload effect:** lower and mid-range payloads <7000 Kg are most reliable.
- **Orbit effect:** LEO missions succeed more often than GTO.
- **Booster effect:** Later versions (FT, B4, B5) outperform earlier ones.
- **Geography:** Sites are coastal and near infrastructure, optimizing safety and logistics. Sites closer to the equator (like CCAFS in Florida) tend to have more launches, leveraging Earth's rotational speed for efficiency.
- **Predictive Modeling:** We used models as Logistic Regression, Support Vector Machine (SVM), Decision Tree, K-Nearest Neighbors (KNN). All achieved ~83% test accuracy with 3 false positives. **Decision Tree** performed better only in terms of validation accuracy **0.875**. All models are good at confirming when the Falcon 9 first stage will land successfully, but they are less effective at identifying when it will fail



Section 2

Insights drawn from EDA

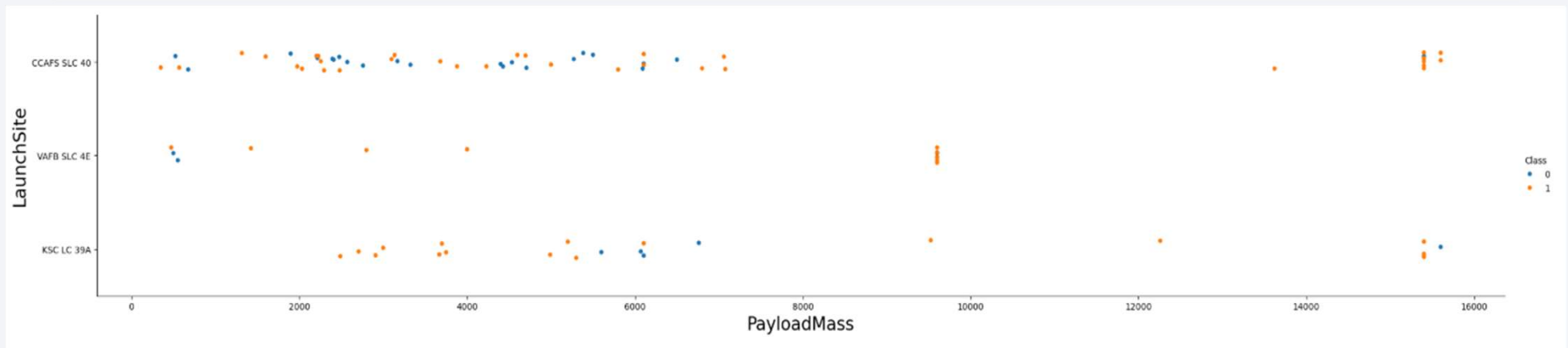
Flight Number vs. Launch Site



- **KSC LC-39A** shows a higher proportion of successful landings (more orange). This site was used at later flights (higher flight numbers).
- **CCAFS SLC-40** has a mix of successes and failures.
- **VAFB SLC-4E** shows fewer launches overall, with more successful landings.

→ This plot demonstrates **how success rates evolved over time at different launch sites**. It highlights SpaceX's learning curve: early missions had more failures, but later missions achieved higher success, with site-specific differences in reliability.

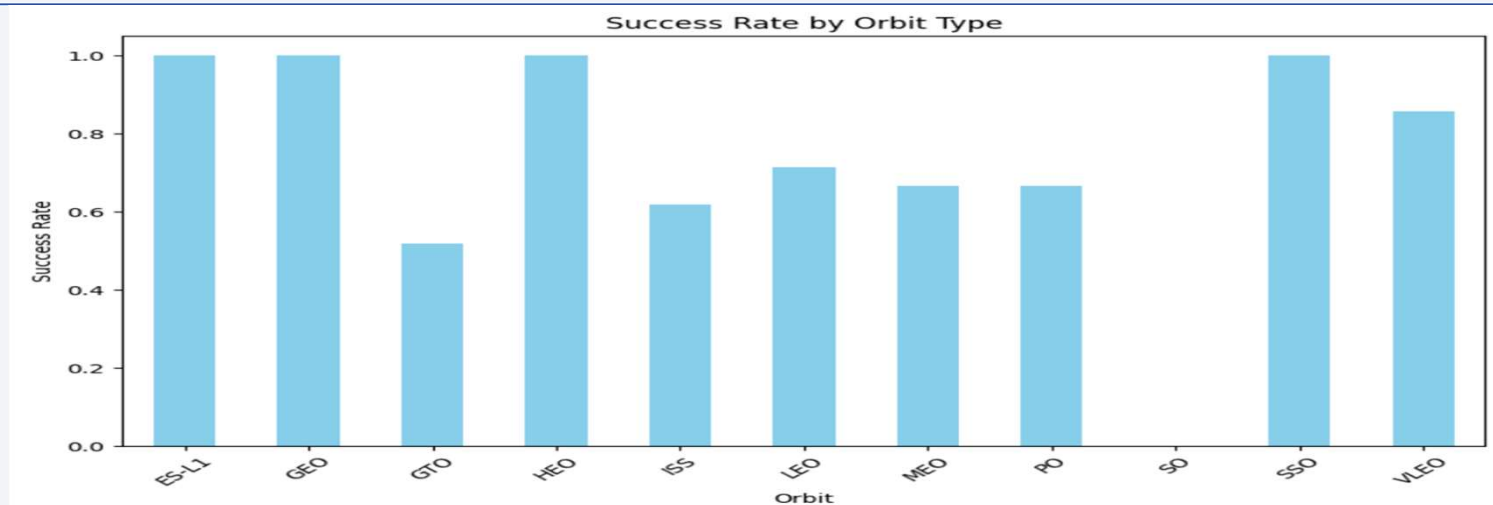
Payload vs. Launch Site



- **VAFB SLC-4E** → Mostly lighter payloads (< 10 000 kg)
- **CCAFS SLC-40 & KSC LC-39A** → Handled heavier payloads, showing broader capability.

→ This plot highlights how **payload mass varies across launch sites and influences success**. VAFB focuses on lighter missions, while CCAFS and KSC manage heavier payloads with higher success rates, reflecting their role in more complex launches.

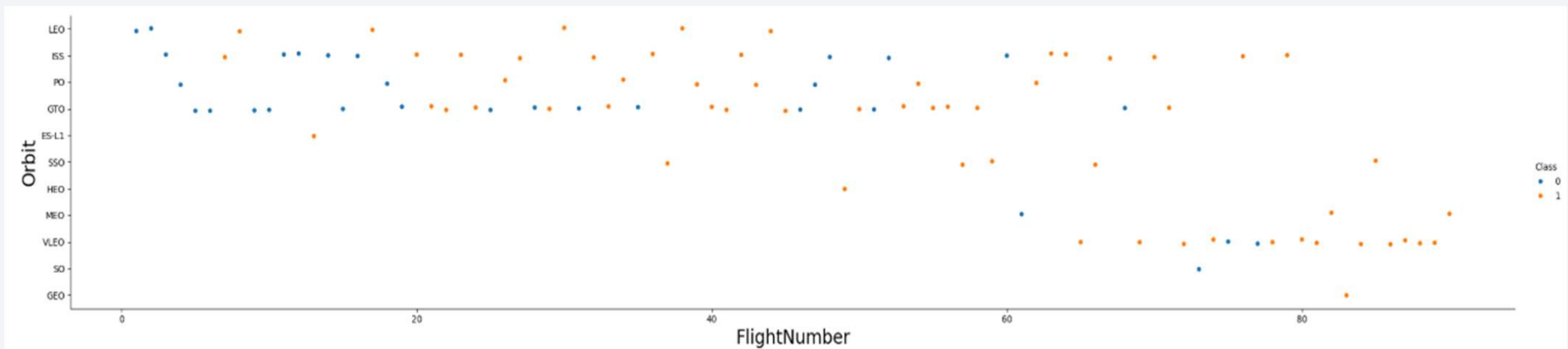
Success Rate vs. Orbit Type



- **High Success Orbits** → ES-L1, GEO, HEO, and SSO show near-perfect success rates (~100%). However, these orbit types have very few missions, so the sample size is limited.
- **Moderate to High Success** → LEO, ISS, PO, MEO, VLEO
- **Lowest Success** → GTO has the lowest success rate (<0.6)

→ Orbit type strongly influences mission success.

Flight Number vs. Orbit Type



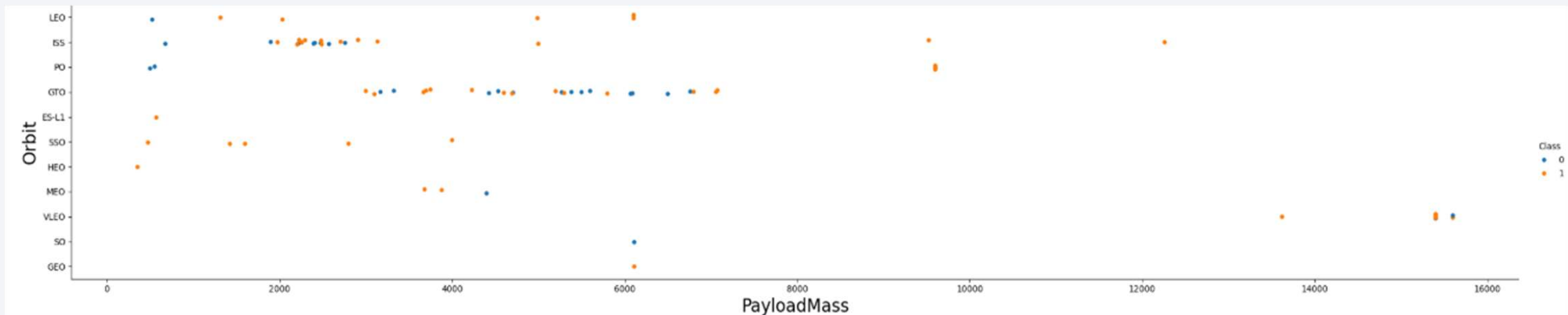
LEO, ISS, PO → Show strong improvement with flight number, many successful landings.

GTO → Remains inconsistent regardless of flight number, highlighting its technical difficulty.

ES-L1, GEO, HEO, SSO, VLEO → Few missions, but most are successful.

→ This plot demonstrates how **experience (flight number)** interacts with **orbit type** to influence success. SpaceX improved steadily in LEO, ISS and PO missions as flight numbers grew, but GTO missions highlight persistent challenges.

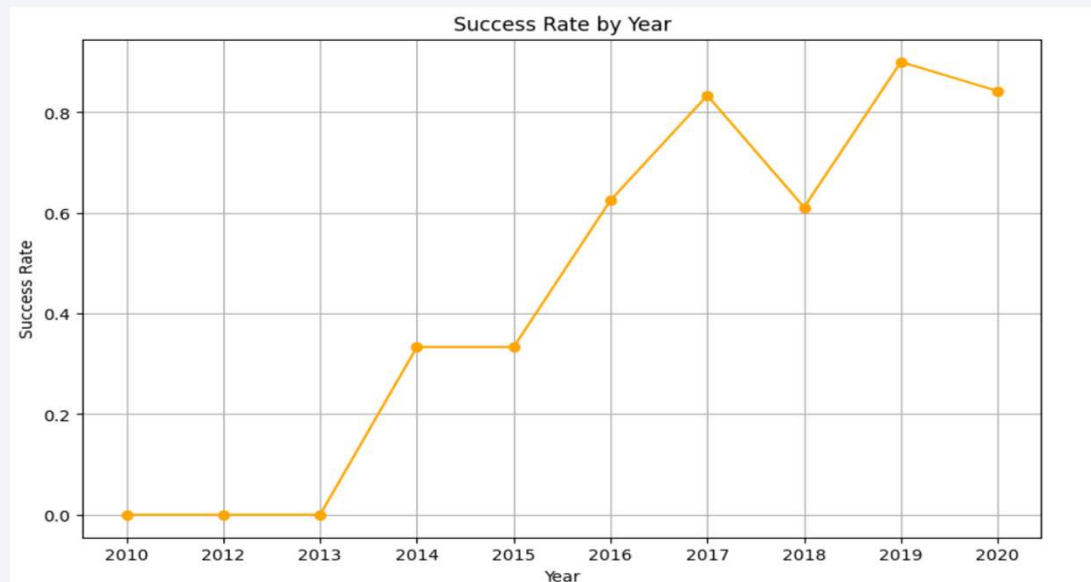
Payload vs. Orbit Type



- **LEO, ISS, PO, VLEO** → Heavy payloads were successfully landed.
- **GTO** → Outcomes are mixed. Both successes and failures occur across payload ranges.
- **ES-L1, SSO, HEO** → Few missions, but most are successful (lighter payload mass).
- **MEO, GEO** → Shows moderate to high success, with payloads in mid-range values.

→ This plot highlights how **payload mass interacts with orbit type to influence landing success**. Success is not strictly limited by payload mass.

Launch Success Yearly Trend



- Success rate rose steadily from 2013, peaked around 2017–2019 (~88%), showing technological maturity

All Launch Site Names

- Find the names of the unique launch sites :

```
SELECT DISTINCT "Launch_Site" FROM SPACEXTABLE
```

- CCAFS LC-40
- CCAFS SLC-40
- KSC LC-39A
- VAFB SLC-4E

→ This query extracts the **unique launch site names** used in SpaceX missions from the table SPACEXTABLE. The **DISTINCT** keyword ensures each site appears only once.

Launch Site Names Begin with 'CCA'

- Find 5 records where launch sites begin with 'CCA'

```
SELECT * FROM SPACEXTABLE WHERE "Launch_Site" LIKE 'CCA%' LIMIT 5
```

Date	Time (UTC)	Booster_Version	Launch_Site	Payload	PAYLOAD_MASS_KG	Orbit	Customer	Mission_Outcome	Landing_Outcome
2010-06-04	18:45:00	F9 v1.0 B0003	CCAFS LC-40	Dragon Spacecraft Qualification Unit	0	LEO	SpaceX	Success	Failure (parachute)
2010-12-08	15:43:00	F9 v1.0 B0004	CCAFS LC-40	Dragon demo flight C1, two CubeSats, barrel of Brouere cheese	0	LEO (ISS)	NASA (COTS) NRO	Success	Failure (parachute)
2012-05-22	7:44:00	F9 v1.0 B0005	CCAFS LC-40	Dragon demo flight C2	525	LEO (ISS)	NASA (COTS)	Success	No attempt
2012-10-08	0:35:00	F9 v1.0 B0006	CCAFS LC-40	SpaceX CRS-1	500	LEO (ISS)	NASA (CRS)	Success	No attempt
2013-03-01	15:10:00	F9 v1.0 B0007	CCAFS LC-40	SpaceX CRS-2	677	LEO (ISS)	NASA (CRS)	Success	No attempt

→ LIKE 'CCA%' filters rows where the launch site name begins with 'CCA' and LIMIT to the first 05 matching records. These site corresponds to 'Cape Canaveral Space Force Station' in Florida.

Total Payload Mass

- Calculate the total payload carried by boosters from NASA

```
SELECT SUM("PAYLOAD_MASS__KG_") AS Total_Payload_Mass  
FROM SPACEXTABLE WHERE Customer = 'NASA (CRS)'
```

Total_Payload_Mass = 45596

→ NASA (CRS) are a **series of flights awarded by NASA** for the delivery of cargo and supplies to the International Space Station (ISS)¹. The result shows that across all CRS missions in the dataset, SpaceX delivered a **total payload mass of 45596kg**, reflecting multiple successful resupply missions over the year. NASA's CRS program relied on SpaceX for spaceflight logistics.

¹ https://en.wikipedia.org/wiki/Commercial_Resupply_Services

Average Payload Mass by F9 v1.1

- Calculate the average payload mass carried by booster version F9 v1.1

```
SELECT AVG("PAYLOAD_MASS__KG_") AS Avg_Payload_Mass  
FROM SPACEXTABLE WHERE Booster_Version LIKE 'F9 v1.1%'
```

- **Average Payload Mass = 2534.66**
- The **F9 v1.1 booster version** consistently carried payloads around **2535 kg**. The F9 v1.1 was indeed a transitional design that improved payload capacity compared to F9 v1.0 ².

² https://en.wikipedia.org/wiki/Falcon_9_v1.1#:~:text=Both%20stages%20of%20the%20two,lift%20range%20of%20launch%20systems

First Successful Ground Landing Date

- Find the dates of the first successful landing outcome on ground pad

```
SELECT MIN(Date) AS First_Successful_GroundPad_Landing  
FROM SPACEXTABLE WHERE Landing_Outcome = 'Success (ground pad)'
```

First_Successful_GroundPad_Landing: 2015-12-22

- This date corresponds to the historic **Falcon 9 Flight 20**, when SpaceX successfully landed a booster at Cape Canaveral's Landing Zone 1. It marked a **major milestone in reusable rocket technology**³.

³ https://en.wikipedia.org/wiki/Falcon_9_flight_20

Successful Drone Ship Landing with Payload between 4000 and 6000

- List the names of boosters which have successfully landed on drone ship and had payload mass greater than 4000 but less than 6000

```
SELECT Booster_Version FROM SPACEXTABLE
WHERE Landing_Outcome = 'Success (drone ship)' and 4000 < PAYLOAD_MASS_KG < 6000
```

- The output lists the **booster versions** that meet both conditions.
- These boosters represent **Falcon 9 Full Thrust, Block 4, 5 variants**⁴, which were capable of carrying mid-range payloads and successfully landing on drone ships at sea.

Booster_Version
F9 FT B1021.1
F9 FT B1022
F9 FT B1023.1
F9 FT B1026
F9 FT B1029.1
F9 FT B1021.2
F9 FT B1029.2
F9 FT B1036.1
F9 FT B1038.1
F9 B4 B1041.1
F9 FT B1031.2
F9 B4 B1042.1
F9 B4 B1045.1
F9 B5 B1046.1

⁴ https://starship-spacex.fandom.com/wiki/Falcon_9

Total Number of Successful and Failure Mission Outcomes

- Calculate the total number of successful and failure mission outcomes

```
SELECT Mission_Outcome, COUNT(*) AS Total  
FROM SPACEXTABLE GROUP BY Mission_Outcome
```

Mission_Outcome	Total
Failure (in flight)	1
Success	98
Success	1
Success (payload status unclear)	1

- 98 missions recorded as successful** with 01 mission failure in flight, showing SpaceX's high reliability rate. There are also two special cases: one additional "Success" entry (likely a data duplication or variant in labeling) and one "Success (payload status unclear)" where the booster landed but payload status is unclear.

Boosters Carried Maximum Payload

- List the names of the booster which have carried the maximum payload mass

```
SELECT DISTINCT Booster_Version, "PAYLOAD_MASS_KG_" FROM SPACEXTABLE  
WHERE "PAYLOAD_MASS_KG_" = (SELECT MAX("PAYLOAD_MASS_KG_") FROM  
SPACEXTABLE)
```

- The query identifies boosters that carried the maximum payload Mass of 15 600 kg, and multiple F9 B5 achieved this. These reusable boosters (e.g. B1048, 1049, etc.) seems to be the most powerful.

Booster_Version	PAYLOAD_MASS_KG_
F9 B5 B1048.4	15600
F9 B5 B1049.4	15600
F9 B5 B1051.3	15600
F9 B5 B1056.4	15600
F9 B5 B1048.5	15600
F9 B5 B1051.4	15600
F9 B5 B1049.5	15600
F9 B5 B1060.2	15600
F9 B5 B1058.3	15600
F9 B5 B1051.6	15600
F9 B5 B1060.3	15600
F9 B5 B1049.7	15600

2015 Launch Records

- List the failed landing_outcomes in drone ship, their booster versions, and launch site names for in year 2015:

```
SELECT substr(Date, 6, 2) AS Month, Booster_Version, Launch_Site, Landing_Outcome FROM  
SPACEXTABLE
```

```
WHERE Landing_Outcome ='Failure (drone ship)' AND substr(Date, 0, 5) ='2015'
```

Month	Booster_Version	Launch_Site	Landing_Outcome
01	F9 v1.1 B1012	CCAFS LC-40	Failure (drone ship)
04	F9 v1.1 B1015	CCAFS LC-40	Failure (drone ship)

- 02 missions match:**
 - Jan 2015 → **Booster F9 v1.1 B1012, launched from Cape Canaveral (LC-40).**
 - April 2015 → **Booster F9 v1.1 B1015, also from Cape Canaveral (LC-40).**

These failures on drone ships were crucial learning steps that paved the way for later successful recoveries at sea.

Rank Landing Outcomes Between 2010-06-04 and 2017-03-20

- Rank the count of landing outcomes (such as Failure (drone ship) or Success (ground pad)) between the date 2010-06-04 and 2017-03-20, in descending order:

```
SELECT Landing_Outcome, COUNT(*) AS Outcome_Count FROM SPACEXTABLE  
WHERE Landing_Outcome IN ('Failure (drone ship)', 'Success (ground pad)') AND DATE BETWEEN  
'2010-06-04' AND '2017-03-20' GROUP BY Landing_Outcome ORDER BY Outcome_Count DESC;
```

Landing_Outcome	Outcome_Count
Failure (drone ship)	5
Success (ground pad)	3

→ Failure on drone ships were more frequent (5 times) between 2015-01-10 and 2016-06-15. Successful ground pad landings occurred 3 times, starting with the historic Falcon 9 Flight 20 on December 22, 2015. These repeated failures highlight the experimental phase of landing on drone ship which was technically more challenging than ground landing.

Section 3

Launch Sites Proximities Analysis

Geographical Distribution of SpaceX Launch Sites in USA



- **Highlighted Locations:** Orange labels with dark circle mark major launch sites in California and Florida.
 - **California (VAFB SLC-4E)** → Refers to Vandenberg Space Force Base, used for polar orbit launches.⁵
 - **Florida (CCAFS LC-40, SLC-40, KSC LC-39A)** → Refers to Cape Canaveral Air Force Station (CCAFS), and Kennedy Space Center Launch Complex (KSC LC 39A). CCAFS SLC-40 initially opened as Launch Complex 40 (LC-40)⁶. These sites are ideal for equatorial and geostationary launches⁷. Equator launches leverage Earth's rotational speed to improve efficiency⁸.

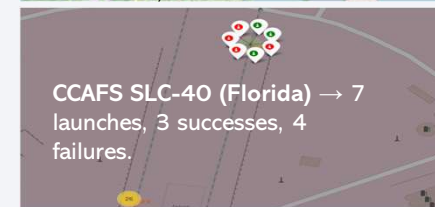
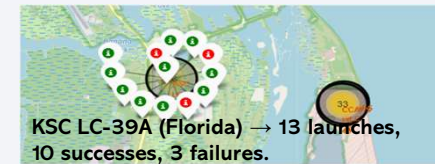
⁵ https://en.wikipedia.org/wiki/Vandenberg_Space_Force_Base

⁶ https://en.wikipedia.org/wiki/Cape_Canaveral_Space_Launch_Complex_40

⁷ <https://www.youtube.com/watch?v=Ej1QUK1m94I&t=12s>

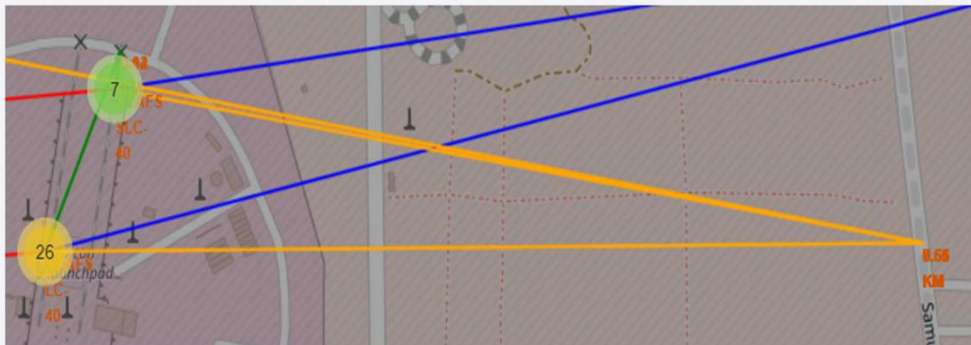
⁸ <https://www.vaia.com/en-us/textbooks/physics/conceptual-physics-12-edition/chapter-10/problem-87-why-are-satellites-normally-sent-into-orbit-by-fi/>

Color-Labeled SpaceX Launch Outcome



- **Florida sites (CCAFS LC-40 & KSC LC-39A)** dominate in launch volume, reflecting their strategic location for equatorial and geostationary missions.
- **KSC LC-39A** stands out as the most reliable pad, while **CCAFS LC-40** shows the challenges of early Falcon 9 development with many failures. **CCAFS SLC-40** has the fewer number of launches with balanced record (3 successes, 4 failures).
- **VAFB SLC-4E** demonstrates SpaceX's capability to launch into polar orbits, though with a lower success rate.

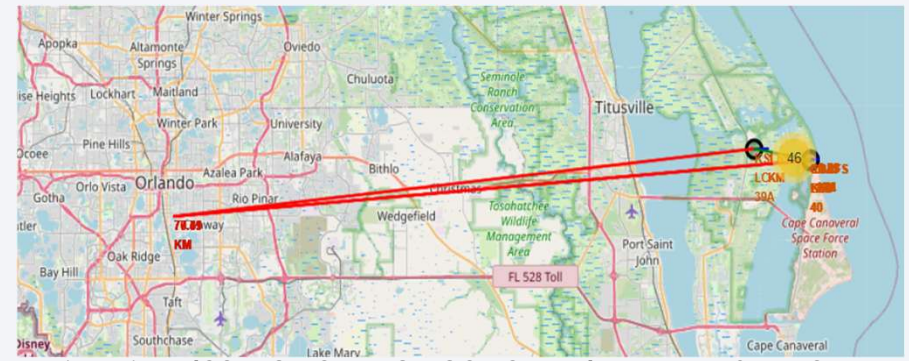
Launch Site Proximity to Coastline and Infrastructure



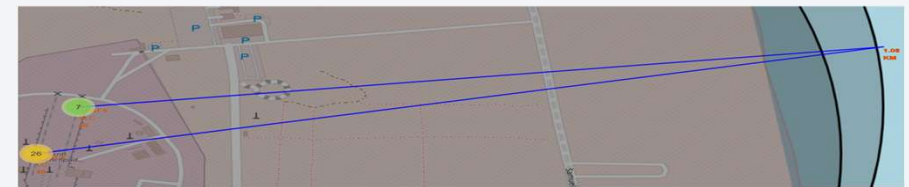
CCAFS LC-40, CCAFS SLC-40: ~ 0.65 km from Highways (orange line)



KSC LC-39A: ~ 0.72 km from railways (green line)



Florida sites (CCAFS LC-40 & KSC LC-39A): ~ 79km from Orlando



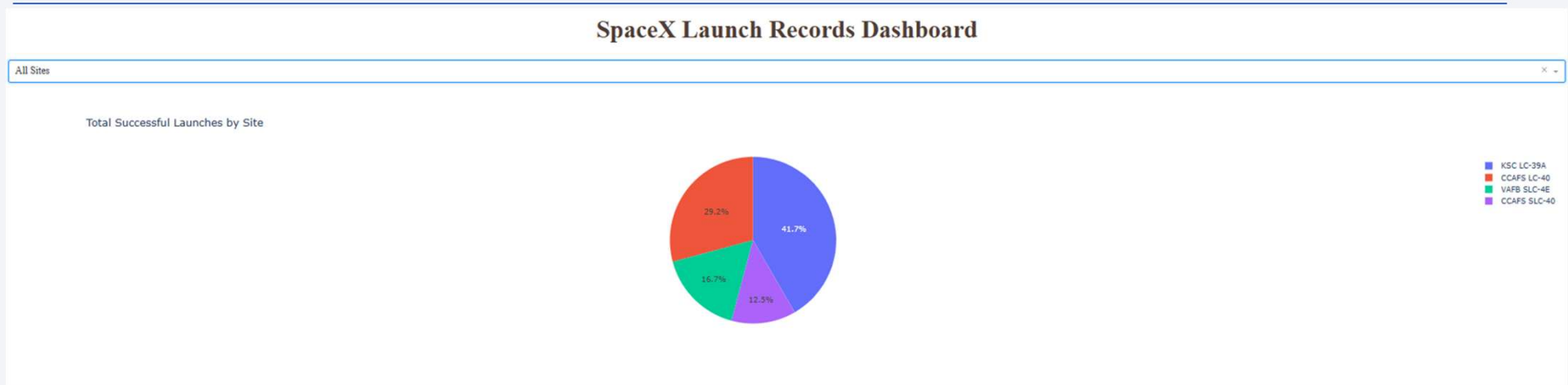
CCAFS LC-40, CCAFS SLC-40: ~ 1,08 km from Coastlines

- Launch sites are located far from cities, along the coast, and close to infrastructure, optimizing safety and logistics.

Section 4

Build a Dashboard with Plotly Dash

Total Successful Launches for All Sites



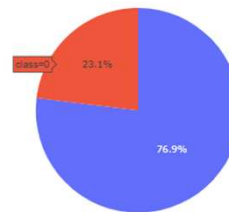
- The dashboard highlights that **LC-39A is SpaceX's most successful launch site with 41.7%**, followed by **LC-40 with 29.2%**, while **VAFB SLC-4E with 16.7%** plays a smaller role for polar orbit missions and finally **SLC-40 with 12.5%**.
- **Florida sites (LC-39A, LC-40, SLC-40) represent over 80% of successful launches**, confirming Cape Canaveral/KSC as the primary hub for equatorial and geostationary missions.

Success vs Failure for Site KSC LC-39

SpaceX Launch Records Dashboard

KSC LC-39A

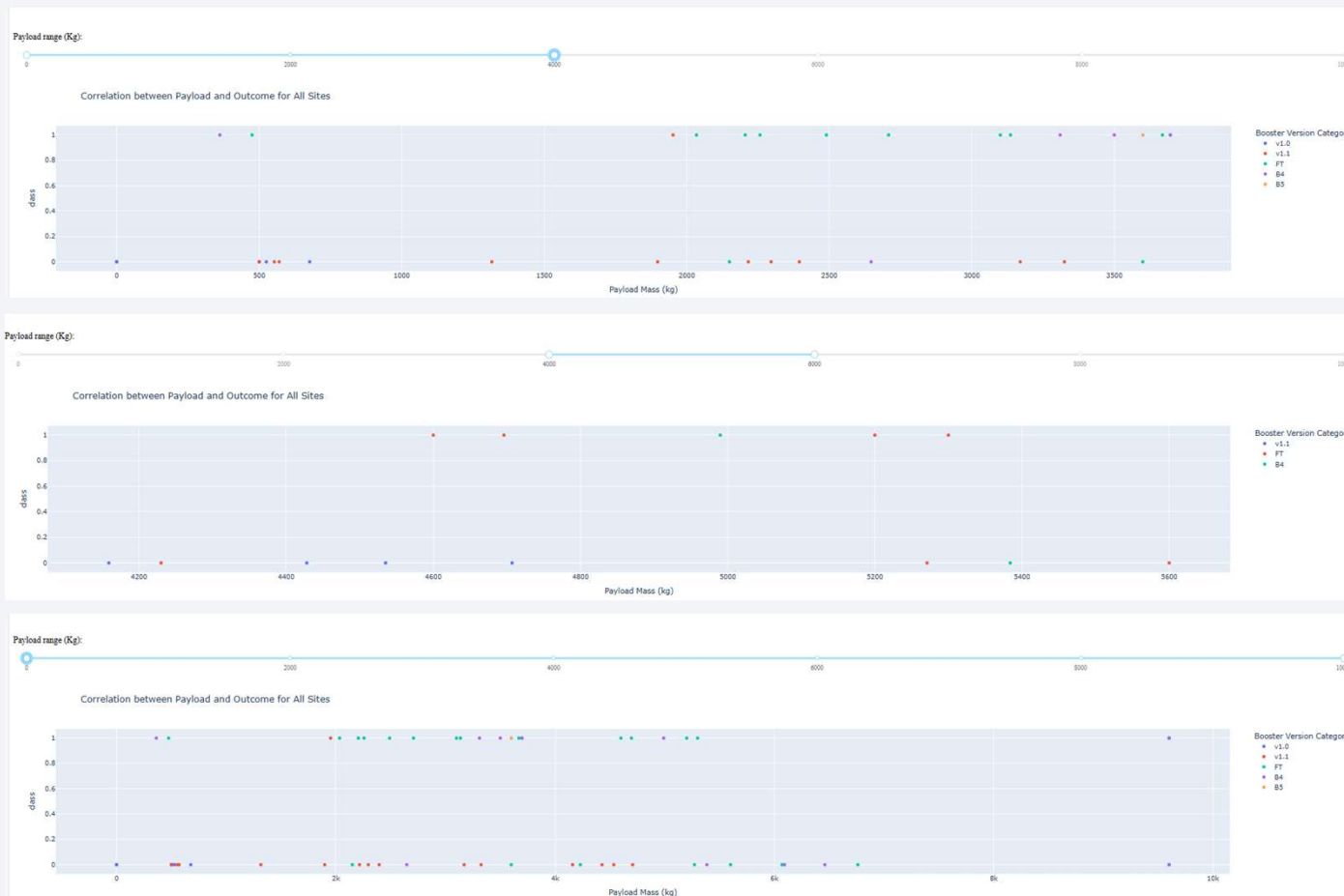
Success vs Failure for site KSC LC-39A



- **KSC LC-39A** shows a strong **success rate of 76.9% Vs 23.1% failure** confirming it as one of SpaceX's most reliable and frequently used pads.
- **KSC LC-39A** is historically significant (Apollo, Shuttle) and now central to **SpaceX Falcon 9 and Falcon Heavy launches**⁹.

⁹ https://www.wikiwand.com/en/articles/Kennedy_Space_Center_Launch_Pad_39A

Correlation Between Payload Mass and Launch Outcomes by Booster Version for all sites



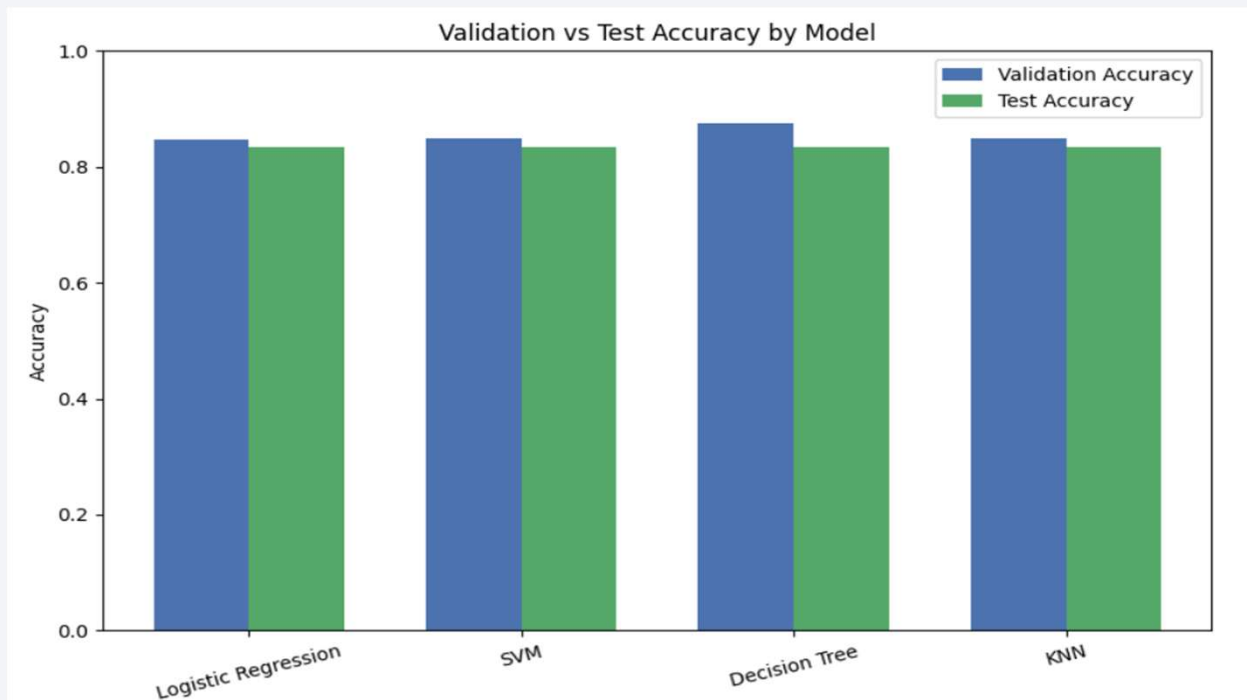
- **Lower payloads (< 4,000 kg):** highest successful launches especially for FT, B4 and B5 (B5 - few data). Most failure for v1.0 and v1.1.
- **Mid-range payloads (4,000 – 6,000 kg):** both FT and B4 boosters experienced more successes than failures.
- **Heavy payloads (> 6,000 kg):** Fewer launches overall with 3 failures and 1 success for B4.

→ The scatter plot shows that newer booster versions (FT, B4 and B5) achieve the highest success rates for lower and mid-range payload mass.

Section 5

Predictive Analysis (Classification)

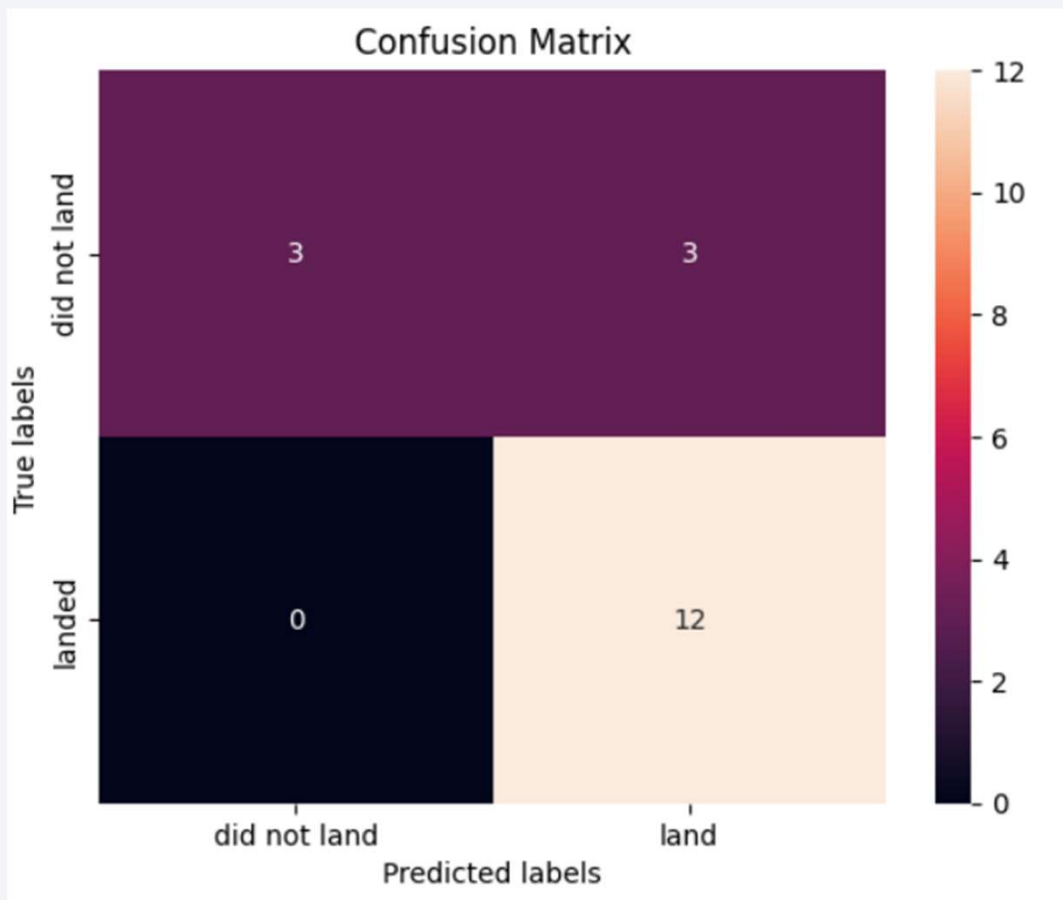
Classification Accuracy



- **Test Accuracy:** All models achieve the same higher accuracy **~0.833**.
- **Validation Accuracy:**
 - **Logistic Regression** → Acc ~0.846
 - **SVM** → Acc ~0.848
 - **Decision Tree** → Acc ~0.875
 - **KNN** → Acc ~0.848

→ Decision Tree is “better” only in terms of validation accuracy (0.875). It seemed promising during training, but in practice it does not outperform the other models on new data.

Confusion Matrix



- All models show the same Confusion Matrix.
 - All models are **perfect at predicting successful landings (12/12 correct)**. This explains why the overall accuracy is relatively high (0.8333).
 - All models struggle to predict True negative (50% correctly predicted – 3 correct, 3 wrong).
- All models are **good at confirming when the Falcon 9 first stage will land successfully**, but they are **less effective at identifying when it will fail**.

Conclusions

- The project demonstrates that predictive models can reliably identify successful Falcon 9 first-stage landings, achieving ~83% accuracy. While the Decision Tree showed the best validation accuracy, all models generalized equally on test data.
- The analysis highlights that **SpaceX's technological progress, newer booster versions, and specific launch sites (notably KSC LC-39A)** are key drivers of success. However, models remain less effective at predicting failures, reflecting the inherent complexity of rocket landings.
- Overall, the study confirms that reusable rockets significantly reduce launch costs and that data-driven approaches can support competitive bidding in the aerospace industry.

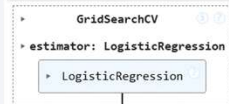
Appendix

TASK 4

Create a logistic regression object then create a GridSearchCV object `logreg_cv` with `cv = 10`. Fit the object to find the best parameters from the dictionary `parameters`.

```
parameters = {'C':[0.01,0.1,1],
              'penalty':['l2'],
              'solver':['lbfgs']}
```

```
parameters = {'C':[0.01,0.1,1], 'penalty':['l2'], 'solver':['lbfgs']}# L1 Lasso L2 ridge
lr=LogisticRegression(max_iter=5000)
logreg_cv = GridSearchCV(lr, parameters, cv=10)
logreg_cv.fit(X_train, Y_train)
```



We output the `GridSearchCV` object for logistic regression. We display the best parameters using the data attribute `best_params_` and the accuracy on the validation data using the data attribute `best_score_`.

```
print("tuned hyperparameters :(best parameters) ",logreg_cv.best_params_)
print("accuracy :",logreg_cv.best_score_)
```

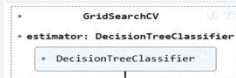
```
tuned hyperparameters :(best parameters) {'C': 0.01, 'penalty': 'l2', 'solver': 'lbfgs'}
accuracy : 0.8464285714285713
```

TASK 8

Create a decision tree classifier object then create a GridSearchCV object `tree_cv` with `cv = 10`. Fit the object to find the best parameters from the dictionary `parameters`.

```
parameters = {'criterion': ['gini', 'entropy'],
              'splitter': ['best', 'random'],
              'max_depth': [2*n for n in range(1,10)],
              #'max_features': ['auto', 'sqrt'],
              'max_features': ['sqrt', 'log2', None], # corrected no auto
              'min_samples_leaf': [1, 2, 4],
              'min_samples_split': [2, 5, 10]}
```

```
tree = DecisionTreeClassifier()
tree_cv=GridSearchCV(tree, parameters, cv=10)
tree_cv.fit(X_train, Y_train)
```



```
print("tuned hyperparameters :(best parameters) ",tree_cv.best_params_)
print("accuracy :",tree_cv.best_score_)
```

```
tuned hyperparameters :(best parameters) {'criterion': 'gini', 'max_depth': 4, 'max_features': 'sqrt', 'min_samples_leaf': 2, 'min_samples_split': 10, 'splitter': 'random'}
accuracy : 0.875
```

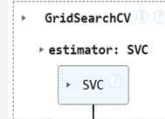
TASK 6

Create a support vector machine object then create a GridSearchCV object `svm_cv` with `cv = 10`. Fit the object to find the best parameters from the dictionary `parameters`.

```
parameters = {'kernel':('linear', 'rbf','poly','rbf', 'sigmoid'),
              'C': np.logspace(-3, 3, 5),
              'gamma':np.logspace(-3, 3, 5)}
```

```
svm = SVC()
```

```
svm_cv=GridSearchCV(svm, parameters, cv=10)
svm_cv.fit(X_train, Y_train)
```



```
print("tuned hyperparameters :(best parameters) ",svm_cv.best_params_)
print("accuracy :",svm_cv.best_score_)
```

```
tuned hyperparameters :(best parameters) {'C': 1.0, 'gamma': 0.03162277660168379, 'kernel': 'sigmoid'}
accuracy : 0.8482142857142856
```

TASK 10

Create a k nearest neighbors object then create a GridSearchCV object `knn_cv` with `cv = 10`. Fit the object to find the best parameters from the dictionary `parameters`.

```
parameters = {'n_neighbors': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
              'algorithm': ['auto', 'ball_tree', 'kd_tree', 'brute'],
              'p': [1,2]}
```

```
KNN = KNeighborsClassifier()
```

```
knn_cv=GridSearchCV(KNN, parameters, cv=10)
knn_cv.fit(X_train, Y_train)
```



```
print("tuned hyperparameters :(best parameters) ",knn_cv.best_params_)
print("accuracy :",knn_cv.best_score_)
```

```
tuned hyperparameters :(best parameters) {'algorithm': 'auto', 'n_neighbors': 10, 'p': 1}
accuracy : 0.8482142857142858
```

Thank you!

